

Módulo 1: Metodologias Ágeis

E-Book - Apostila

Boas vindas!



Boas-vindas à trilha de aprendizagem “Métodos Ágeis”!

Esta trilha é composta por quatro módulos que te convidam a mergulhar no universo das abordagens ágeis, desde seus fundamentos até sua aplicação concreta em diversos contextos profissionais.

Ao longo de quatro módulos, você irá compreender como os métodos ágeis se diferenciam dos modelos tradicionais, explorar frameworks como Scrum, Kanban, XP e Lean, aprender a utilizar ferramentas práticas como Trello, Jira e Miro, e descobrir como cultivar uma cultura organizacional centrada em colaboração, autonomia e melhoria contínua.

Com videoaulas envolventes, estudos de caso e atividades desafiadoras, esta trilha foi pensada para desenvolver seu mindset ágil e prepará-lo para aplicar esses conhecimentos em projetos reais, com segurança e estratégia. Ao final, você também será orientado sobre certificações que podem impulsionar sua carreira na área.

Bons estudos!

Sobre o TIC em trilhas

Iniciativa em parceria com TIC em Trilhas e parte integrante do projeto Residência em TIC 01, Residência em TIC 02 e Residência em TIC 21. Projetos apoiados pelo Ministério da Ciência, Tecnologia e Inovações, com recursos da Lei nº 8.248/91, sendo uma iniciativa do Ministério da Ciência, Tecnologia e Inovações e Softex no âmbito do Programa MCTI Futuro.



Objetivos de aprendizagem

Ao realizar todas as leituras e atividades propostas no módulo você será capaz de:

- Reconhecer as principais características das metodologias de desenvolvimento tradicionais e ágeis, identificando suas diferenças e aplicações práticas.
- Compreender o contexto de surgimento das metodologias ágeis, relacionando-o com as limitações dos modelos tradicionais de desenvolvimento.
- Identificar e explicar os quatro valores centrais do Manifesto Ágil, compreendendo sua relevância na condução de projetos ágeis.
- Interpretar os 12 princípios do Manifesto Ágil e refletir sobre sua aplicação em diferentes cenários profissionais.

Introdução



Freepik, 2025.

No mundo da tecnologia, as metodologias de desenvolvimento de software são essenciais para o planejamento, gerenciamento e execução de projetos. É a partir delas que definimos estruturas e processos que guiarão as equipes ao longo do desenvolvimento, desde a concepção da ideia até a entrega do software final. Nesse primeiro módulo da trilha vamos saber como surgiram as metodologias de desenvolvimento de software e conhecer um documento essencial para as metodologias ágeis: O Manifesto Ágil.

1. Metodologias de desenvolvimento

Você já pensou em todas as fases e atividades que devemos seguir para construir um software?



Você provavelmente vai pensar que é necessário fazer o levantamento de requisitos do software, codificar o produto e realizar testes antes da entrega.

Mas você pensou sobre a necessidade de construção de protótipos para validação do projeto? Ou de construção de diagramas da UML para embasamento da fase de codificação?



A realidade é que temos muito mais etapas e atividades que devem ser seguidas para que consigamos entregar o melhor software possível. Mesmo com equipes altamente qualificadas, desenvolver um software não é uma tarefa fácil e requer muitas etapas. Um erro pode ser crucial e simplesmente transformar o projeto em um grande estorvo.

Quando analisamos a área de software, vemos que no início não havia nenhum padrão de desenvolvimento. Cada empresa (ou até mesmo equipe) desenvolvia os softwares de forma diferente, sem seguir nenhum método. Isso continuou por décadas, até que no fim da década de 60, ocorreu o que foi chamado de "Crise do software". Nesse momento da história da computação, a indústria começou a notar que a falta de uma metodologia estava atrapalhando a entrega de softwares. A complexidade dos softwares aumentava cada dia mais, os projetos frequentemente levavam mais tempo e custavam mais do que havia sido previsto, os softwares continham muitos erros e não atendiam às necessidades dos usuários. Além de todos esses problemas, as equipes não tinham o mínimo de documentação necessária para gerenciar o software após a entrega.

Imagine: como podemos dar manutenção ou criar versões de um software que não foi documentado ou em que a equipe não seguiu nenhuma metodologia?



No meio disso tudo, a área de software começou a buscar por métodos de gerir melhor os projetos. Assim nasceram as metodologias de desenvolvimento de software, visando organizar todo processo de desenvolvimento. Qualquer metodologia de desenvolvimento de software, seja tradicional ou ágil, tem o mesmo objetivo: guiar o trabalho das equipes, garantindo que o produto atenda às necessidades do cliente e seja entregue dentro do prazo e orçamento estipulados.

- **Metodologias Tradicionais**

As metodologias tradicionais não são o objetivo da nossa trilha, mas não podemos falar sobre criação das metodologias ágeis sem antes passar por elas. As primeiras metodologias criadas para lidar com a Crise do Software são as chamadas metodologias tradicionais de desenvolvimento.

Por serem as primeiras, elas têm uma série de características bem fáceis de identificar:

São abordagens estruturadas de forma sequencial, onde cada parte de planejamento do projeto é bem detalhado e cada fase acontece de forma linear, ou seja, uma fase apenas começa quando a anterior é concluída.

Assim, todo o processo de desenvolvimento é composto por fases bem definidas, o que (em teoria) facilita o gerenciamento do projeto.

Porém na realidade, esse tipo de metodologia exige algo bem difícil de conseguir:

Todos os requisitos devem ser bem conhecidos e estáveis logo no início do projeto.

Veja bem: se a ideia é realizar cada fase de uma vez e de forma rígida, nesse tipo de modelo qualquer repetição de atividades (como por exemplo o levantamento de requisitos) não é viável. Ou seja, metodologias tradicionais são pouco flexíveis em relação a mudanças do projeto.

Outro ponto negativo muito indicado é:

Nesse tipo de metodologia, a ênfase muito grande na documentação e no planejamento pode acarretar atrasos na codificação do projeto.

O modelo tradicional mais famoso é o modelo Cascata. Esse modelo, também chamado de modelo clássico de desenvolvimento, foi a primeira metodologia de desenvolvimento de software criada.



Imagem criada por IA

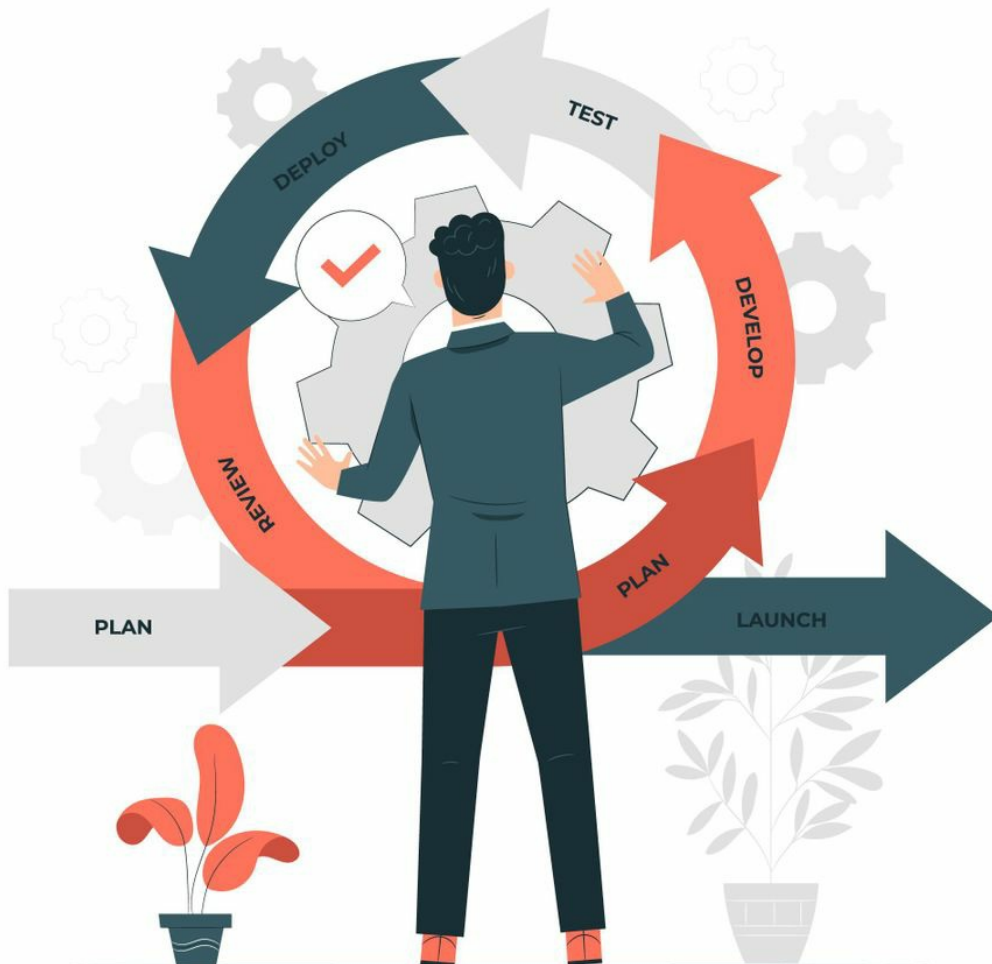
Nele, cada fase (por exemplo, levantamento de requisitos, implementação, testes etc.) é feita de forma sequencial e sistemática. Isso quer dizer que cada fase é rigidamente definida, realizada e concluída antes que a equipe possa passar para a próxima fase.

Como esse modelo é bem fechado e rígido, isso implica que o cliente já saiba de antemão todos os requisitos do projeto, já que dificilmente há brecha para mudanças (ou seja: retrabalho). As principais críticas ao modelo indicam que os projetos raramente seguem esse fluxo sequencial e que o cliente dificilmente sabe todos os requisitos logo no início do projeto. Além disso, não há feedback entre as fases e um atraso em uma fase é igual à atraso em todas as fases posteriores. Além do mais, o cliente só recebe o software no final e nesse meio tempo os requisitos e a própria empresa podem ter passado por mudanças que impactaram no software que estava sendo desenvolvido.

O modelo cascata serviu como base para criação de todos os modelos que temos. Porém, apesar dos avanços trazidos pelas metodologias tradicionais, a rigidez com que o modelo exige que o software seja desenvolvido e todos os outros problemas citados acima, fizeram que com o tempo a área de tecnologia entendesse que era necessária uma mudança para que os softwares pudessem atender às expectativas dos clientes. Esses problemas levaram à adoção de uma nova forma de desenvolvimento de software: as metodologias ágeis.

- **Metodologias Ágeis**

Todo avanço que as metodologias tradicionais trouxeram para o desenvolvimento de software, estas ainda apresentavam falhas. Buscando uma nova forma de desenvolver software, na década de 1990 surgiu o movimento de desenvolvimento ágil.



Freepik, 2025.

Esse movimento que vinha como resposta aos problemas das metodologias tradicionais, indicava que o desenvolvimento de software deveria ser flexível, com foco na colaboração e na entrega contínua. Assim, o objetivo principal das metodologias ágeis é o desenvolvimento rápido de software, através de ciclos curtos e iterativos.

Cada ciclo gera pequenas partes do software, permitindo que o cliente tenha acesso a pedaços do software com frequência. Isso facilita a adaptação às mudanças nos requisitos e no ambiente de negócios, além de permitir feedback frequente do cliente, gerando um produto mais alinhado com as expectativas do cliente.

Segundo Pressman (2016):

“A engenharia de software ágil combina filosofia com um conjunto de princípios de desenvolvimento. A filosofia defende a **satisfação do cliente** e a **entrega incremental antecipada**; **equipes de projeto pequenas** e altamente motivadas; métodos informais; **artefatos de engenharia de software mínimos**; e acima de tudo, **simplicidade no desenvolvimento em geral**. Tanto engenheiro quanto cliente têm o mesmo parecer: o único artefato realmente importante consiste em um “incremento de software” operacional que seja entregue, adequadamente, na data combinada”.

Destacamos em negrito todos os pontos que nos ajudam a entender como funcionam as metodologias ágeis. É muito importante para esse movimento que o cliente esteja satisfeito e isso é facilitado e a equipe consegue fazer entregas pequenas e rápidas de software.

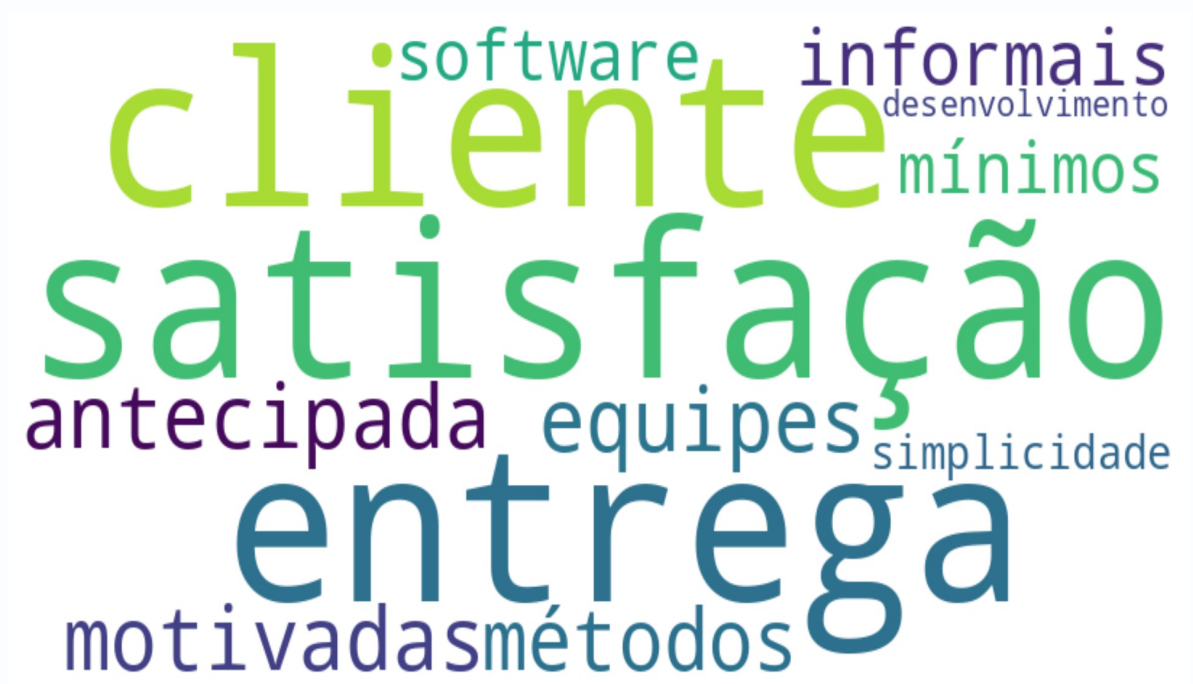


Imagem criada por IA

Para isso, é necessário simplicidade no desenvolvimento, ou a equipe não conseguirá cumprir com o acordado com o cliente, já que normalmente equipes ágeis são pequenas (por volta de 10 integrantes).

Mas como equipes tão pequenas conseguem realizar entregas frequentes?



Para as metodologias ágeis, os artefatos (documentação) devem ser mínimos. Isso ajuda a equipe a focar no que realmente importa: a entrega do software funcional.

Porém, devemos ter cuidado ao analisar essa afirmação. Muita gente confunde metodologias ágeis com pressa, e isso não poderia estar mais equivocado! Pressa tem um sentido negativo, conota imprudência. Quando pensamos em alguém que está sempre com pressa, logo pensamos que essa pessoa é desorganizada e negligente.

No desenvolvimento de software, pressa implica pular etapas o que pode levar a um software ineficiente.

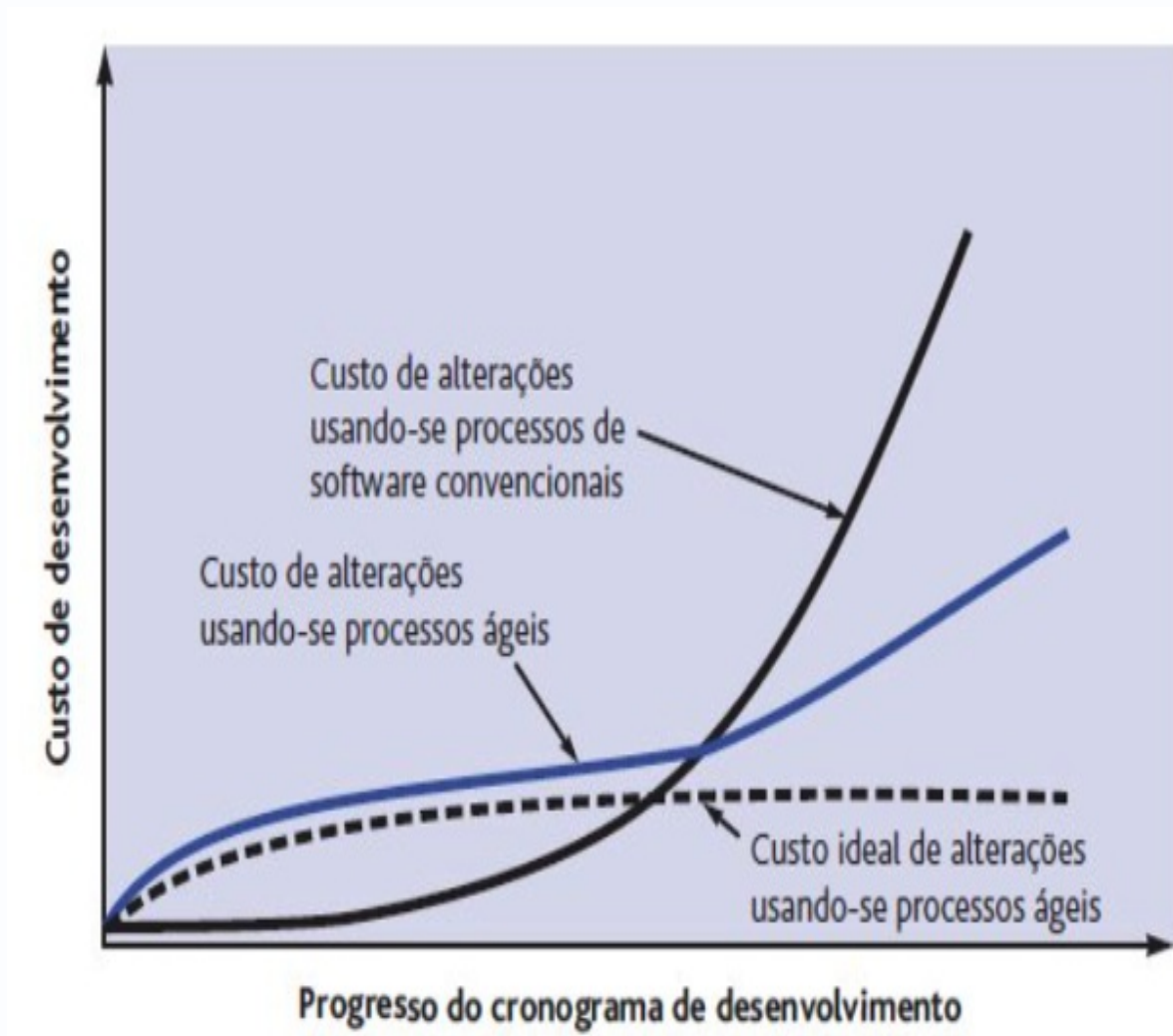
Agilidade por outro lado traz um sentido positivo. Quando dizemos que alguém é ágil, normalmente estamos a elogiando, dizendo que essa pessoa é rápida, porém competente.

No desenvolvimento de software, entendemos que equipes ágeis são equipes rápidas em entregar o software, porém cumprem todo o necessário para entregar o melhor produto possível.

E como as metodologias ágeis ajudam as equipes a controlarem os custos de mudanças no desenvolvimento?



Na figura a seguir, Pressman (2016) indica a curva de custo de mudanças no desenvolvimento de software quando utilizamos metodologias tradicionais e ágeis.



Fonte: Pressman, 2016.

Nos modelos tradicionais, os custos das mudanças aumentam consideravelmente de acordo com o avanço do projeto. Como vimos, nas metodologias tradicionais, o software não recebe feedback do cliente, por isso, as chances de solicitações de correções ou manutenção no software são altas no final. Isso piora a situação, aumentando o custo do software quanto mais avançado estiver o projeto. Já nas metodologias ágeis, como a equipe além de ter feedback busca atender às mudanças durante o projeto, a curva de custos é menor, ou seja, a solução de problemas durante o projeto diminui o custo das mudanças no final.

Para o Pressman (2016), podemos até mesmo adaptar métodos tradicionais para a agilidade:

“A agilidade pode ser aplicada a qualquer processo de software. Entretanto, para alcançá-la, é essencial que o processo seja projetado de modo que a equipe possa adaptar e alinhar (racionalizar) tarefas; possa conduzir o planejamento, compreendendo a fluidez de uma metodologia de desenvolvimento ágil; possa eliminar tudo, exceto os artefatos essenciais, conservando-os enxutos; e enfatize a estratégia de entrega incremental, conseguindo entregar ao cliente, o mais rapidamente possível, o software operacional para o tipo de produto e ambiente operacional”.

Agora que você sabe como funcionam os métodos ágeis de desenvolvimento de software no geral, vamos falar do documento que deu origem a tudo: o manifesto ágil.

2. Manifesto Ágil: princípios e valores



Freepik, 2025.

Em 2001, descontentes com os problemas que as metodologias tradicionais traziam para o desenvolvimento de software, um grupo de desenvolvedores se reuniu para discutir alternativas às metodologias tradicionais. Nesse grupo de 17 desenvolvedores, havia criadores do Scrum, do XP e vários programadores com carreira consolidada na área. Essas pessoas já desenvolviam software de forma diferente ao desenvolvimento tradicional e buscavam também a divulgação de boas práticas que eram exercidas dentro das suas empresas. Dessa reunião, saiu um documento assinado pelos 17 participantes, que indicava os valores e princípios essenciais para o desenvolvimento de software no início dos anos 2000.

Esse documento, chamado de Manifesto ágil, mudou completamente a área de desenvolvimento de software, ao apresentar uma filosofia sobre o que realmente era importante naquele momento da indústria. O manifesto veio para apontar novos rumos no desenvolvimento de software, potencializando os projetos e resultados a partir de uma equipe ágil. Para além disso, os criadores buscavam que erros fossem diagnosticados e corrigidos mais rapidamente, que a equipe tivesse maior liberdade de trabalho, e que o cliente fizesse parte do desenvolvimento de software.

Como tudo na Engenharia de Software, o objetivo principal era:

Entregar um software de qualidade no menor espaço de tempo possível.

Para que isso fosse alcançado, estabeleceram 4 valores e 12 princípios. No manifesto ágil há a indicação de que indivíduos e suas interações, software em funcionamento, colaboração, e resposta a mudanças eram o de mais importante que havia no desenvolvimento de software. Tudo diferente disso (ou seja: processos e ferramentas, documentação abrangente, negociação de contratos e plano), seria algo secundário e complementar ao processo de desenvolvimento.



Adaptado de: Sankhya, 2022.

Vamos analisar melhor cada um desses valores?

Clique nas abas para entender cada valor.

**Valor 1: Os indivíduos e suas iterações acima de procedimentos e ferramentas.**

O manifesto enfatiza que as pessoas e como elas interagem são mais importantes do que os processos rígidos e as ferramentas que elas usam. Quem desenvolve software são pessoas e quem usa o software são pessoas. A comunicação eficaz e a colaboração entre as equipes são fundamentais para o sucesso do projeto. Assim, as interações entre os envolvidos devem ser baseadas em uma rede de comunicação de qualidade que auxilie o desenvolvimento do software.

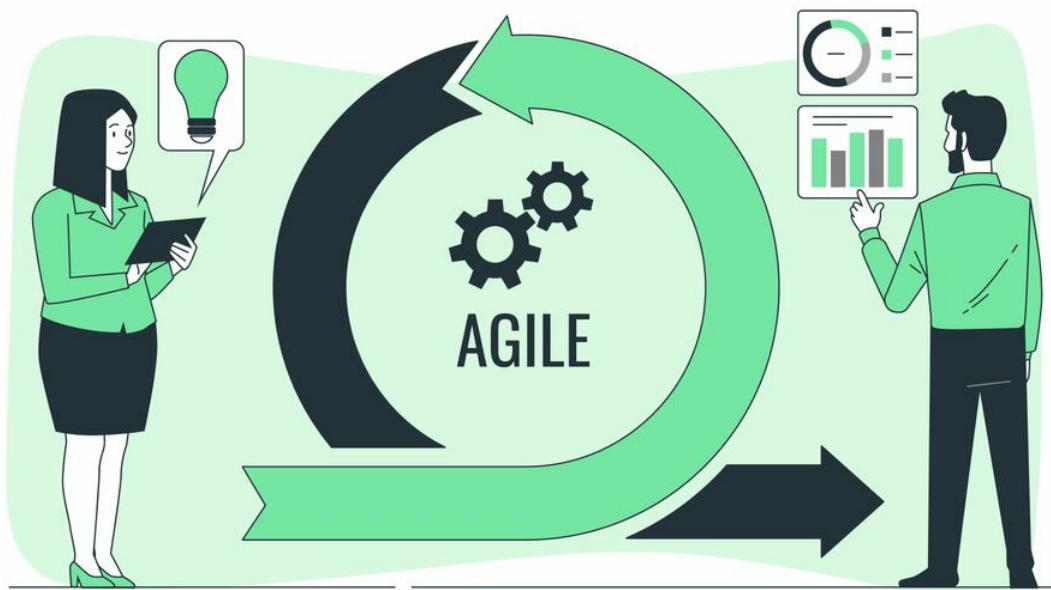
**Valor 2: O funcionamento do software acima da documentação.**

Devemos sempre nos lembrar que para o cliente o mais importante é o software funcionando. Além disso, o software em funcionamento é o indicador de que o projeto foi realizado com sucesso. Documentar o software é um passo importante no desenvolvimento, porém muitas equipes podem se perder em meio a tanta documentação. Em vez de investir muito tempo em documentações detalhadas, o foco deve ser na entrega de software que realmente funcione e atenda às necessidades do cliente. Ou seja, a documentação deve ser adequada ao contexto e ao que é necessário para o sucesso do projeto.



Valor 3: A colaboração com o cliente acima de negociação e contrato.

O manifesto destaca a importância de trabalhar com o cliente durante todo o processo de desenvolvimento, ajustando o projeto às suas necessidades e feedback. O cliente pode (e deve) contribuir para o desenvolvimento de um software de qualidade, por isso, a colaboração entre equipe de desenvolvimento e cliente pode ser a chave de sucesso de um software. Negociar um contrato pode ser necessário, mas a colaboração contínua é mais importante.

**Valor 4: A capacidade de resposta a mudanças acima de um plano pré-estabelecido.**

A flexibilidade para mudar e se adaptar é fundamental no desenvolvimento ágil. A equipe deve sempre estar preparada e atenta para possíveis mudanças no software. Essas mudanças podem vir por exemplo a partir de feedbacks do cliente ou de uma análise de mercado feita pela própria equipe de desenvolvimento. Em vez de seguir um plano fixo, a equipe deve estar disposta a se adaptar conforme surgem novas informações ou mudanças nas necessidades do cliente.

Além dos valores, o Manifesto indica doze princípios do desenvolvimento ágil. Clique nas abas para entender cada um.

1. Nossa maior prioridade é satisfazer o cliente, por meio da entrega adiantada e contínua de um software de valor.

O objetivo deste princípio é garantir que a necessidade do cliente seja satisfeita. A equipe deve entregar software funcional e de valor com frequência, desde o início até o final do projeto. Isso ajuda a criar um relacionamento de confiança e a garantir que o produto atenda às expectativas do cliente.

2. Aceitar mudanças de requisitos, mesmo no fim do desenvolvimento, pois os processos ágeis se adequam às necessidades do cliente.

O manifesto destaca que as mudanças no escopo do projeto devem ser vistas como uma oportunidade e não um problema para a equipe de desenvolvimento. O projeto deve sempre ser flexível, permitindo ajustes conforme as necessidades do cliente mudam.

3. Entregar uma versão do software funcionando com frequência, na escala de semanas até meses, com preferência para períodos mais curtos.

A entrega regular de software é fundamental para o sucesso da aplicação de uma metodologia ágil. A entrega deve sempre ser incremental, pois isso ajuda a obter feedback rápido, a identificar problemas cedo e a garantir que o projeto permaneça no caminho certo.

4. Profissionais de negócios e desenvolvedores devem trabalhar em conjunto e diariamente, durante todo o andamento do projeto.

Colaboração e comunicação são a chave das metodologias ágeis. A comunicação constante e a colaboração entre todos os participantes do projeto (equipe de desenvolvimento, clientes e usuários) são essenciais para o sucesso do software. A troca contínua de informações também ajuda a alinhar as expectativas dos envolvidos.

5. Construir projetos ao redor de indivíduos motivados, oferecendo a eles o ambiente e suporte necessário, e confiar que farão seu trabalho.

As equipes ágeis devem ser formadas por pessoas capacitadas e motivadas. Além disso, é necessário que tenham as ferramentas e o ambiente necessário para fazer um bom trabalho de acordo com os valores e princípios ágeis. Quando isso é respeitado, as chances do projeto ser bem-sucedido são maiores.

6. O método mais eficiente e eficaz de transmitir informações para um time de desenvolvimento é uma conversa cara a cara.

A interação é parte importante do desenvolvimento de software. Para o manifesto ágil, ela deve ser direta por meio de reuniões (presenciais ou virtuais), pois isso facilita a comunicação clara e rápida. Isso também ajuda a resolver os problemas do projeto mais rapidamente e a garantir que todos estejam na mesma página.

7. Um software funcional é a medida primária de progresso.

O objetivo final de todo o projeto de desenvolvimento é entregar um software que funcione e que atenda ao cliente. Uma forma de medir o progresso do projeto é pela entrega de funcionalidades úteis. Isso pode ser mais valioso do que métricas como número de horas trabalhadas ou quantidade de código feito.

8. Processos ágeis promovem um ambiente sustentável em que os patrocinadores, desenvolvedores e usuários são capazes de manter passos constantes.

Projetos de software tendem a ser longos e por isso é necessário evitar a sobrecarga da equipe. A equipe deve ser capaz de trabalhar em um ritmo constante e sustentável, evitando exaustão e mantendo a produtividade ao longo do tempo. Isso é essencial para o sucesso do projeto a longo prazo

9. Atenção contínua à excelência técnica e bom design para aumentar a agilidade.

A equipe deve investir na qualidade técnica e no design sólido do software. Boas práticas de codificação, testes e design resultarão em software mais flexível, fácil de manter e de adaptar às mudanças.

10. Simplicidade em primeiro lugar, como a arte de otimizar o trabalho.

Com entregas frequentes, a simplicidade é uma necessidade no desenvolvimento ágil. Deve-se evitar funcionalidades desnecessárias e simplificar processos e decisões, para que a equipe tenha tempo de manter o foco no que realmente importa.

11. As melhores arquiteturas, requisitos e designs emergem de times auto-organizáveis.

As equipes ágeis devem ser autônomas, ou seja, devem ter liberdade para tomar decisões sobre como melhor organizar o trabalho, como arquitetar o sistema e como abordar o design. Isso fomenta a inovação e a criatividade, resultando em soluções mais eficazes.

12. Em intervalos regulares, o time deve pensar em como aumentar sua efetividade e ajustar seu comportamento de acordo.

As equipes devem se reunir regularmente para refletir sobre o que está funcionando bem e o que pode ser melhorado. Essa prática de autoavaliação contínua ajuda a equipe a ajustar seus processos e práticas para se tornar mais eficaz e eficiente ao longo do tempo.

O manifesto é a base para qualquer método ágil e é fundamental para o desenvolvimento de software de forma flexível, colaborativa e eficiente. Esse documento incentiva as mudanças rápidas, o feedback constante e o foco da equipe na entrega de valor real para o cliente.

Refletindo

Gostou de saber mais sobre a história das metodologias de desenvolvimento? Vamos pensar em um exemplo sobre quando devemos escolher entre uma metodologia tradicional ou ágil?

Imagine que você é um gerente de projetos e deve selecionar a melhor forma de trabalho da equipe. Como você escolheria entre metodologias tradicionais e metodologias ágeis? Pare alguns minutos, reflita sobre isso e anote suas reflexões.

Para escolher entre uma metodologia tradicional ou uma metodologia ágil, você pode começar respondendo as seguintes perguntas:



1. Qual o tipo de entrega desejado pelo cliente?



2. Os requisitos são estáveis e conhecidos?



3. O cliente terá uma participação ativa no projeto?



4. Como é a equipe de desenvolvimento ?



5. A equipe se adapta facilmente a mudanças?

Você também pode analisar como é o tipo de liderança da e o gerenciamento da empresa nessa definição. Há muitos itens a serem analisados nesse tipo de escolha, mas todos eles passam pelo tipo de projeto e tipo de equipe.

Se ainda ficou com dúvida, fique tranquilo, no módulo 3 voltaremos a conversar sobre esse assunto.

No próximo módulo saberemos mais sobre as metodologias ágeis mais importantes. Te vejo lá!

Referências bibliográficas

Metodologias ágeis: conceito, o que mudou e próximos passos. **Sankhya**, 2022. Disponível em: <<https://www.sankhya.com.br/blog/metodologias-ageis/>>. Acesso em: 03 mar. de 2025.

MONTEIRO, Aguiar; SOUZA, Aislan Rafael Rodrigues; LEAL, Sérgio Sierro. Como estimar um Software? Métricas para aferição de Esforço, Prazo e Custo de um Produto de Software. In: TELES, Ariel Soares; CALÇADA, Dario Brito; VERAS, Necio de Lima (org.). Livro de Minicursos da ERCMAPI 2020. 1. ed. Porto Alegre: SBC, 2020. p.169-187. Disponível em: <https://books-sol.sbc.org.br/index.php/sbc/catalog/book/48/222/460-1>. Acesso em 15 de março de 2025.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software. 8ª edição. [S.I.]. McGraw Hill Brasil, 2016.

